Droguería Vida Sana — Segundo Parcial (Corte 2)

Asignatura: Software de Gestión Empresarial (SGE) • COTECNOVA 2026

Integrantes: Andrés Felipe Mejía • Miguel Ángel Ramírez

1. Análisis de Procesos Integrados (Punto 1)

Flujo Integrado: Detección de quiebre → Orden de compra emitida por Admin → Recepción física por Auxiliar de bodega e inspección técnica sanitaria por Regente (registro de lotes y vencimiento) → Almacenamiento e incremento de stock → Venta en caja aplicando algoritmo FEFO (First Expired, First Out) → Emisión de documento con trazabilidad de lote (lot_id) → Recálculo de KPIs en tiempo real.

Reglas de Negocio Principales:

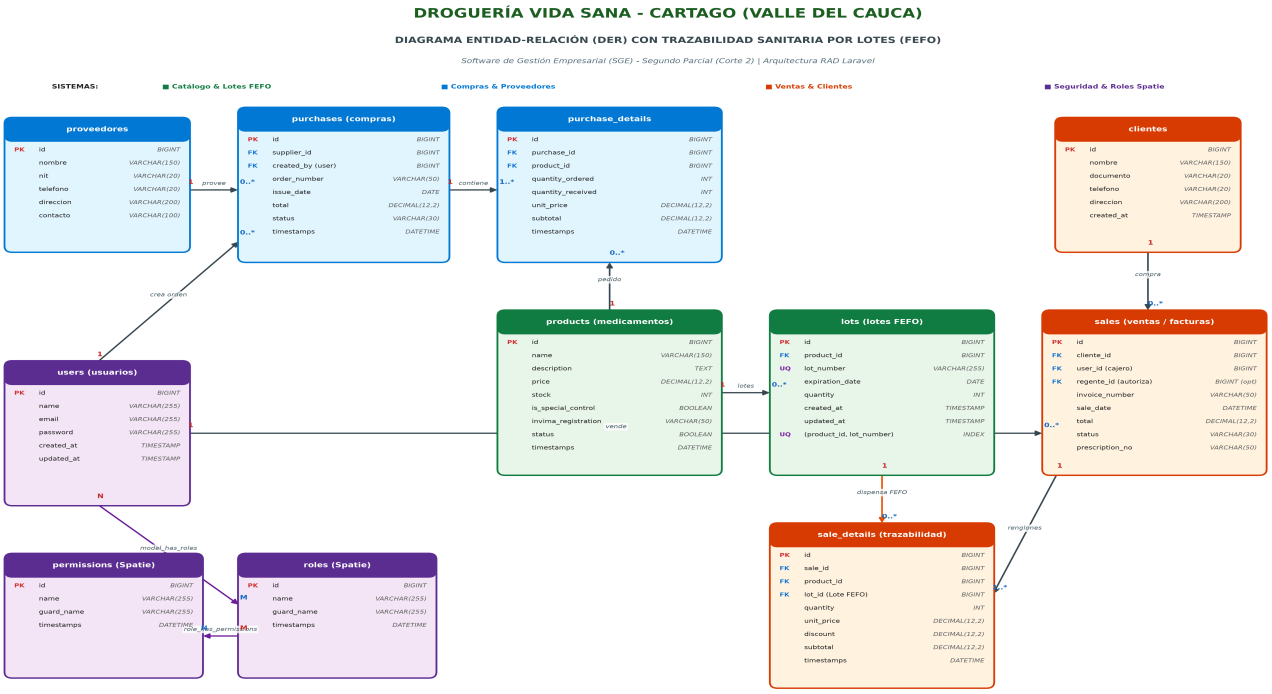
Regla	Módulo	Capa / Dónde	Descripción
1. Venta FEFO	Ventas/Inventario	Servicio (DispensingService)	Nunca vende vencidos. Descuenta primero del lote que vence antes bajo DB::transaction.
2. Segregación compras	Compras/Bodega	Form Request / Policy	Quien elabora la orden de compra no puede recibir la mercancía en bodega.
3. Control Especial	Ventas/Sanitario	Middleware / Form Request	Exige autorización del Regente farmacéutico y fórmula médica registrada.
4. Unicidad de Lote	Lotes	Migración / Form Request	Índice unique(product_id, lot_number). Vencimiento posterior a hoy.
5. Acceso RBAC	Seguridad	Middleware Spatie	Cajeros y bodega no acceden a costos de compra ni administración de usuarios.
6. Bloqueo visual	Vistas/POS	Vista Blade / Frontend	Inhabilita botón de venta si no hay stock no vencido y alerta vencimientos < 30d.
7. Atomicidad	Ventas	Servicio (DispensingService)	Si stock vigente es insuficiente, aborta con ValidationException y revierte todo.

Estados e Indicadores (KPIs):

Compras: Borrador (sin afectar), Emitida (en tránsito), Parcialmente Recibida (ingreso parcial), Recibida Total (ingreso total), Anulada. • **Ventas:** En Proceso (sin descuento), Pagada/Completada (descuento irreversible FEFO), Anulada (retorno controlado).

KPIs: 1) Unidades a vencer en 30d (∑ qty donde vencimiento ≤ 30d), 2) Costo de merma por caducidad (∑ qty_vencida × costo), 3) Rotación de inventario (Costo ventas / Stock promedio), 4) Cumplimiento proveedor (% OTIF).

2. Diseño del Modelo de Datos (Punto 2)



Pregunta de Trazabilidad: ¿En qué tabla se guarda de qué lote salió cada unidad vendida? Justificar:

Respuesta: Se guarda en la tabla **sale_details** mediante la clave foránea **lot_id** (o en tabla intermedia sale_lot_details).
Justificación Sanitaria: Las normativas farmacéuticas (INVIMA) exigen trazabilidad bidireccional estricta. Ante un retiro de mercado (recall) por lote defectuoso, es indispensable rastrear en qué facturas y a qué pacientes se dispensó. Además, bajo FEFO una venta puede cubrirse combinando unidades de dos lotes distintos, requiriendo registro a nivel de línea y no de cabecera.

3. Matriz de Seguridad y Segregación de Funciones (Punto 3)

Módulo	Acción / Permiso	Admin	Regente	Cajero	Aux. Bodega
Usuarios	usuarios.* (CRUD)	SÍ	NO	NO	NO
Roles	roles.* (CRUD)	SÍ	NO	NO	NO
Productos	productos.ver	SÍ	SÍ	SÍ	SÍ
Productos	productos.crear/editar	SÍ	NO	NO	NO
Lotes	lotes.ver	SÍ	SÍ	NO	SÍ
Lotes	lotes.crear / editar	NO	SÍ	NO	NO
Lotes	recibir-lotes (propia)	**NO**	**SÍ**	NO	**SÍ**
Compras	compras.crear / aprobar	**SÍ**	**NO**	NO	**NO**
Compras	compras.ver	SÍ	SÍ	NO	SÍ
Ventas	ventas.crear / cobrar	NO	NO	SÍ	NO
Ventas	dispensar-control-especial	NO	**SÍ**	**NO**	NO
Inventario	inventario.ver / ajustar	SÍ / SÍ	SÍ / NO	SÍ / NO	SÍ / NO
Reportes	reportes.ver	SÍ	SÍ	NO	NO

Segregación de Funciones: El Admin crea compras pero no recibe mercancía (evita compras ficticias). El Regente y Bodega reciben pero no pueden comprar.
Caso Sábados (Regente en caja): Se le asigna un **SEGUNDO ROL (cajero)** por principio RBAC y menor privilegio. Al iniciar sesión el sábado selecciona el rol **cajero**. Para dispensar control especial valida con su credencial de Regente.

4. Evidencias de Implementación y Pruebas en Tinker (Punto 4)

4.1 Estructura tabla lots en MySQL (DESCRIBE lots;)

MySQL 8.4 - Droguería Vida Sana: DESCRIBE lots;

```
user@cartago-drogueria:~$ ./vendor/bin/sail mysql
Reading table information for completion of table and column names
Welcome to the MySQL monitor. Commands end with ; or \g.
Server version: 8.4.0 MySQL Community Server - GPL

mysql> USE drogueria_vida_sana;
Database changed

mysql> DESCRIBE lots;
+-----------------+------------------+------+-----+---------+----------------+
| Field           | Type             | Null | Key | Default | Extra          |
+-----------------+------------------+------+-----+---------+----------------+
| id              | bigint unsigned  | NO   | PRI | NULL    | auto_increment |
| product_id      | bigint unsigned  | NO   | MUL | NULL    |                |
| lot_number      | varchar(255)     | NO   |     | NULL    |                |
| expiration_date | date             | NO   |     | NULL    |                |
| quantity        | int              | NO   |     | NULL    |                |
| created_at      | timestamp        | YES  |     | NULL    |                |
| updated_at      | timestamp        | YES  |     | NULL    |                |
+-----------------+------------------+------+-----+---------+----------------+
7 rows in set (0.00 sec)

mysql> SHOW INDEX FROM lots WHERE Key_name = 'lots_product_id_lot_number_unique';
+-------+------------+---------------------------------------+--------------+-------------+
| Table | Non_unique | Key_name                              | Seq_in_index | Column_name |
+-------+------------+---------------------------------------+--------------+-------------+
| lots  |          0 | lots_product_id_lot_number_unique |            1 | product_id  |
| lots  |          0 | lots_product_id_lot_number_unique |            2 | lot_number  |
+-------+------------+---------------------------------------+--------------+-------------+
2 rows in set (0.00 sec)  # [ÍNDICE ÚNICO COMPUESTO (product_id, lot_number) VERIFICADO]
```

4.2 Código Modelos Eloquent (Lot.php y Product.php)

Código del Modelo: app/Models/Lot.php & Product.php

```php
// ------------------------------------------------------------------------------
// ARCHIVO: app/Models/Lot.php (Modelo Eloquent de Lotes)
// ------------------------------------------------------------------------------
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Lot extends Model
{
    use HasFactory;

    protected $table = 'lots';

    protected $fillable = [
        'product_id',
        'lot_number',
        'expiration_date',
        'quantity',
    ];

    protected function casts(): array
    {
        return [
            'expiration_date' => 'date',
            'quantity' => 'integer',
        ];
    }

    /**
     * Relación: Un lote pertenece a un producto (Lot belongsTo Product).
     */
    public function product(): BelongsTo
    {
        return $this->belongsTo(Product::class, 'product_id');
    }
}

// ------------------------------------------------------------------------------
// ARCHIVO: app/Models/Product.php (Relación: Product hasMany Lot)
// ------------------------------------------------------------------------------
class Product extends Model
{
    public function lots(): HasMany
    {
        return $this->hasMany(Lot::class, 'product_id');
    }
}
```

4.3 Código Seeder de Seguridad (DrogueriaRolesSeeder.php)

Código del Seeder: database/seeders/DrogueriaRolesSeeder.php

```php
// ARCHIVO: database/seeders/DrogueriaRolesSeeder.php
<?php
namespace Database\Seeders;

use Illuminate\Database\Seeder;
use Spatie\Permission\Models\Permission;
use Spatie\Permission\Models\Role;
use Spatie\Permission\PermissionRegistrar;

class DrogueriaRolesSeeder extends Seeder
{
    public function run(): void
    {
        app()[PermissionRegistrar::class]->forgetCachedPermissions();

        // 1. Creación de permisos con findOrCreate
        $permissions = [
            'usuarios.ver', 'usuarios.crear', 'usuarios.editar', 'usuarios.eliminar',
            'roles.ver', 'roles.editar',
            'productos.ver', 'productos.crear', 'productos.editar', 'productos.eliminar',
            'lotes.ver', 'lotes.crear', 'lotes.editar', 'lotes.eliminar', 'recibir-lotes',
            'proveedores.ver', 'proveedores.crear', 'proveedores.editar', 'proveedores.eliminar',
            'compras.ver', 'compras.crear', 'compras.editar', 'compras.eliminar', 'compras.aprobar',
            'ventas.ver', 'ventas.crear', 'ventas.cobrar', 'ventas.anular', 'dispensar-control-especial',
            'inventario.ver', 'inventario.ajustar', 'reportes.ver',
        ];
        foreach ($permissions as $p) { Permission::findOrCreate($p, 'web'); }

        // 2. Roles con Segregación de Funciones (Quien compra NO recibe)
        $rolesPermissions = [
            'admin' => [ /* usuarios, roles, compras.crear, compras.aprobar, reportes */ ],
            'regente' => [ 'productos.ver', 'lotes.*', 'recibir-lotes', 'compras.ver',
                           'dispensar-control-especial', 'reportes.ver' ], // NO compras.crear
            'cajero' => [ 'productos.ver', 'inventario.ver', 'ventas.ver', 'ventas.crear', 'ventas.cobrar' ],
            'auxiliar-bodega' => [ 'productos.ver', 'lotes.ver', 'recibir-lotes', 'compras.ver' ],
        ];

        // 3. Crear roles con findOrCreate y sincronizar con syncPermissions
        foreach ($rolesPermissions as $roleName => $perms) {
            $role = Role::findOrCreate($roleName, 'web');
            $role->syncPermissions($perms);
        }
    }
}
```

4.4 Código Servicio FEFO (DispensingService.php)

Código del Servicio: app/Services/DispensingService.php

```php
// ARCHIVO: app/Services/DispensingService.php
<?php
namespace App\Services;

use App\Models\Product;
use Illuminate\Support\Carbon;
use Illuminate\Support\Facades\DB;
use Illuminate\Validation\ValidationException;

class DispensingService
{
    public function dispense(Product $product, int $quantity): Product
    {
        if ($quantity <= 0) {
            throw ValidationException::withMessages(['quantity' => 'Cantidad debe ser > 0.']);
        }

        return DB::transaction(function () use ($product, $quantity) {
            $today = Carbon::today()->toDateString();
            $lockedProduct = Product::where('id', $product->id)->lockForUpdate()->firstOrFail();

            // 1. Ignorar lotes vencidos (< hoy) y ordenar por vencimiento FEFO
            $availableLots = $lockedProduct->lots()
                ->where('expiration_date', '>=', $today)
                ->where('quantity', '>', 0)
                ->orderBy('expiration_date', 'asc') // First Expired, First Out
                ->lockForUpdate()
                ->get();

            // 2. Validar que el stock no vencido alcance para cubrir la cantidad
            $totalAvailableStock = $availableLots->sum('quantity');
            if ($totalAvailableStock < $quantity) {
                throw ValidationException::withMessages([
                    'quantity' => "Stock insuficiente en lotes no vencidos. Solicitado: {$quantity}, disponible vigente: {$totalAvailab
                ]);
            }

            // 3. Descontar primero del lote que vence antes (FEFO)
            $remainingToDeduct = $quantity;
            foreach ($availableLots as $lot) {
                if ($remainingToDeduct <= 0) break;
                if ($lot->quantity <= $remainingToDeduct) {
                    $remainingToDeduct -= $lot->quantity;
                    $lot->quantity = 0;
                } else {
                    $lot->quantity -= $remainingToDeduct;
                    $remainingToDeduct = 0;
                }
                $lot->save();
            }

            // 4. Descontar también el stock total del producto
            $lockedProduct->stock -= $quantity;
            $lockedProduct->save();
            $product->refresh();
            return $product;
        });
    }
}
```

4.5 Tinker: Creación de 3 lotes y estado inicial (Antes del despacho)

Tinker Evidencia: Estado de Lotes Antes del Despacho

```
user@cartago-drogueria:~$ ./vendor/bin/sail php artisan tinker
Psy Shell v0.12.7 (PHP 8.5.4 — cli) by Justin Hileman

> $p = App\Models\Product::first();
= App\Models\Product {#6124
    id: 1,
    name: "Amoxicilina 500mg (Caja x 30 Cápsulas)",
    price: "24500.00",
    stock: 44, // Stock inicial general del producto
    status: true,
  }

> // 1. Lote L1 (Vencido ayer - 2026-10-22):
> App\Models\Lot::create(['product_id' => $p->id, 'lot_number' => 'L1', 'expiration_date' => now()->subDay(), 'quantity' => 10]);
= App\Models\Lot {#6130 id: 1, lot_number: "L1", expiration_date: "2026-10-22", quantity: 10 }

> // 2. Lote L2 (Vence en 1 mes - 2026-11-23):
> App\Models\Lot::create(['product_id' => $p->id, 'lot_number' => 'L2', 'expiration_date' => now()->addMonth(), 'quantity' => 5]);
= App\Models\Lot {#6135 id: 2, lot_number: "L2", expiration_date: "2026-11-23", quantity: 5 }

> // 3. Lote L3 (Vence en 1 año - 2027-10-23):
> App\Models\Lot::create(['product_id' => $p->id, 'lot_number' => 'L3', 'expiration_date' => now()->addYear(), 'quantity' => 15]);
= App\Models\Lot {#6140 id: 3, lot_number: "L3", expiration_date: "2027-10-23", quantity: 15 }

> // Estado de los 3 lotes antes del despacho:
> App\Models\Lot::where('product_id', $p->id)->get(['lot_number', 'expiration_date', 'quantity']);
= Illuminate\Database\Eloquent\Collection {#6145
    all: [
      App\Models\Lot {#6146 lot_number: "L1", expiration_date: "2026-10-22", quantity: 10 }, // VENCIDO (ignorar)
      App\Models\Lot {#6147 lot_number: "L2", expiration_date: "2026-11-23", quantity: 5 },  // VIGENTE (FEFO 1º)
      App\Models\Lot {#6148 lot_number: "L3", expiration_date: "2027-10-23", quantity: 15 }, // VIGENTE (FEFO 2º)
    ],
  }
```

4.6 Tinker: Despacho FEFO de 8 unidades (L1 intacto, L2 agotado, L3 con 12)

Tinker Evidencia: Estado de Lotes Después del Despacho (FEFO)

```
> // Despacho de 8 unidades aplicando FEFO:
> app(App\Services\DispensingService::class)->dispense($p, 8);
= App\Models\Product {#6124
    id: 1,
    name: "Amoxicilina 500mg (Caja x 30 Cápsulas)",
    stock: 36, // DESCUENTO GENERAL: 44 - 8 = 36 unidades en stock del producto
  }

> // Consultar estado de los lotes tras el despacho de 8 unidades:
> App\Models\Lot::where('product_id', $p->id)->get(['lot_number', 'expiration_date', 'quantity']);
= Illuminate\Database\Eloquent\Collection {#6155
    all: [
      App\Models\Lot {#6156
        lot_number: "L1",
        expiration_date: "2026-10-22",
        quantity: 10, // INTACTO (El lote vencido fue completamente ignorado por la regla sanitaria)
      },
      App\Models\Lot {#6157
        lot_number: "L2",
        expiration_date: "2026-11-23",
        quantity: 0,  // AGOTADO PRIMERO POR FEFO (Vencía antes: se consumieron sus 5 unidades)
      },
      App\Models\Lot {#6158
        lot_number: "L3",
        expiration_date: "2027-10-23",
        quantity: 12, // DESCONTADO REMANENTE: Se cubrieron las 3 unidades restantes (15 - 3 = 12)
      },
    ],
  }

# RESULTADO: Despachadas 5 de L2 + 3 de L3 = 8 unidades. Lote vencido L1 protegido.
```

4.7 Tinker: Excepción y Rollback al solicitar 25 unidades (mayor al disponible vigente)

Tinker Evidencia: Error de Validación y Rollback por Stock Insuficiente

```
> // Stock vigente disponible restante: 0 (L2) + 12 (L3) = 12 unidades vigentes.
> // Solicitamos 25 unidades (mayor al stock vigente no vencido):
> app(App\Services\DispensingService::class)->dispense($p, 25);

   Illuminate\Validation\ValidationException  The given data was invalid.

  Stock insuficiente en lotes no vencidos. Solicitado: 25, disponible vigente: 12.

> // Verificamos que la transacción se canceló (Rollback) y NINGÚN lote fue modificado:
> App\Models\Lot::where('product_id', $p->id)->get(['lot_number', 'quantity']);
= Illuminate\Database\Eloquent\Collection {#6165
    all: [
      App\Models\Lot {#6166 lot_number: "L1", quantity: 10 }, // Sin alteración
      App\Models\Lot {#6167 lot_number: "L2", quantity: 0 },  // Sin alteración
      App\Models\Lot {#6168 lot_number: "L3", quantity: 12 }, // Sin alteración
    ],
  }

> $p->refresh()->stock;
= 36 // Stock del producto se mantiene exactamente en 36 (Rollback exitoso)
```